systems

MDPI

Article

TimeWeaver: Time-Aware Sequential Recommender System via Dual-Stream Temporal Network

Yang Liu [1,*], Tao Wang [1] and Yan Ma [2,3]

[1] School of Computer and Artificial Intelligence, Nanjing University of Finance and Economics, Nanjing 210023, China; 1120230547@stu.nufe.edu.cn
[2] School of Accounting, Nanjing University of Finance and Economics, Nanjing 210023, China; yanma@nufe.edu.cn
[3] Department of Information Systems and Analytics, National University of Singapore, Singapore 117417, Singapore
* Correspondence: yliu@nufe.edu.cn

Abstract

Recommender systems are data-driven tools designed to assist or automate users' decision-making. With the growing demand of personalized sequential recommendations in business intelligence or e-commerce, effectively capturing temporal information from massive user-sequence data has become a crucial challenge. State-of-the-art attention-based models often struggle to balance performance with computational cost, while traditional convolutional neural networks suffer from limited receptive fields and rigid architectures that inadequately model dynamic user interests. To address these limitations, this paper proposes TimeWeaver, a time-aware dual-stream network for sequential recommendation, whose core innovations comprise three key components. First, it employs a re-parameterized large-kernel convolution to expand the effective receptive field. Second, we design a Time-Aware Augmentation mechanism that integrates inter-event time-interval information into positional encodings of items. This allows it to perceive the temporal dynamics of user behavior. Finally, we propose a dual-stream architecture to jointly capture dependencies across different time scales. The context stream employs a modern Temporal Convolutional Network (TCN) structure to strengthen the memorization of users' medium- and long-term interests. In parallel, the dynamic stream leverages an Exponential Moving Average (EMA) mechanism to weight recent behaviors for sensitively capturing users' immediate interests. This dual-stream design allows TimeWeaver to comprehensively extract both long- and short-term sequential features. Extensive experiments on three public e-commerce datasets demonstrate TimeWeaver's superiority. Compared to the strongest baseline model, TimeWeaver achieves average relative improvements of 4.62%, 9.59%, and 4.59% across all metrics on the Beauty, Sports, and Toys datasets, respectively.

Keywords: sequential recommender system; dual-stream architecture; temporal convolution; time-aware representation

check for updates

Academic Editors: Rui Nunes Cruz and Albérico Travassos Rosário

Received: 20 August 2025
Revised: 22 September 2025
Accepted: 28 September 2025
Published: 29 September 2025

Citation: Liu, Y.; Wang, T.; Ma, Y. TimeWeaver: Time-Aware Sequential Recommender System via Dual-Stream Temporal Network. *Systems* **2025**, *13*, 857. https://doi.org/10.3390/systems13100857

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

As a central component of data-driven marketing decisions, personalized recommender systems are crucial for optimizing the user experience. Amidst widespread digitalization, user behavior data is becoming increasingly complex and dynamic. Sequential Recommender System is a key subfield of personalized recommender systems that aims to accurately predict future user behavior by modeling the temporal dependencies in user

interactions [1]. However, a fundamental challenge in this field is how to precisely capture the evolution of user interests from massive sequential interaction data. This limitation directly impedes the capacity of marketing analytics systems for efficient prediction and real-time intervention.

From a systems thinking perspective, a user's interaction sequence is not merely a linear stream of events but a complex adaptive system. Within this system, a user's decision-making process is often influenced by factors across multiple time scales. This requires a sequential recommender model to adopt a holistic view, enabling it to simultaneously comprehend a user's immediate contextual needs, recent interests, and long-term stable preferences. The complex interplay of these multiple time scales already lies beyond the scope of traditional methods.

Early Markov Chain [2] models laid a foundation for sequential modeling, but their limited ability to capture long-range dependencies makes them insufficient for complex user behavior sequences. In response, deep learning-based models for sequential recommendation have emerged and undergone rapid development. Among these, Recurrent Neural Networks (RNNs) and their variants, such as Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU), were first introduced to capture temporal dependencies [3]. More recently, Transformer-based models, leveraging the self-attention mechanism, have achieved significant success in sequential recommendation due to their powerful parallel processing and long-range dependency modeling capabilities. Concurrently, Convolutional Neural Networks (CNNs) offer another effective approach for sequence modeling, with unique advantages in extracting local features and hierarchical information [4]. Specifically, Temporal Convolutional Networks (TCNs) [5] have shown considerable potential by using causal and dilated convolutions to capture long-term dependencies in time-series data [6].

However, with the growing prominence of Transformer [7] and MLP [8] architectures, the focus on CNNs within the sequential recommendation field has gradually diminished. TCNs, as a convolutional architecture suited for sequential data, can theoretically capture long-term dependencies effectively. This capability stems primarily from their use of dilated convolutions, which allow the receptive field to expand exponentially with network depth [9]. Yet, existing TCN-based models face several challenges in practical applications [10]. First, many TCN models do not fully leverage their potential to expand the receptive field within recommendation scenarios. Traditional fixed-size convolutional kernels and static dilation strategies struggle to flexibly capture dependencies across the diverse time spans found in different user behavior sequences. Second, existing TCN architectures tend to focus on sequence orderliness while failing to explicitly model key temporal attributes, particularly the specific time intervals between interactions. For business intelligence systems, such temporal information is critical for understanding customer behavior and making timely decisions [11]. User behavior is not only influenced by item order but is also intricately linked to the precise timestamps of interactions, the duration of intervals between actions, and the dynamic evolution of user interests across multiple time scales. Overlooking this fine-grained temporal information can lead marketing analytics systems to misinterpret user intent. For instance, a high density of interactions in a short period may indicate strong current interest, whereas an interaction after a long interval could signify a shift in interest or the emergence of a new need, often presenting an opportune moment for marketing interventions.

To address the aforementioned limitations and account for dependencies across different time scales, we propose TimeWeaver, a dual-stream temporal convolutional model designed to differentiate between time scales. The model is composed of two parallel processing streams: a context stream and a dynamic stream. On the one hand, some user interests are persistent and can span multiple short-term interaction clusters, which

requires the model to have the capacity to capture these long-term trends. To achieve this, the context stream incorporates a modern TCN architecture, leveraging its large receptive field to effectively extract stable feature representations. On the other hand, user behavior sequences also contain short-term interests that reflect immediate needs, necessitating a capacity for responsive real-time prediction. For this purpose, we introduce an Exponential Moving Average (EMA) method into the dynamic stream to enhance the model's sensitivity to short-term fluctuations, thereby capturing rapidly changing information from recent user interactions more accurately.

The remainder of this paper is organized as follows. Section 2 reviews the related literature on sequential recommender systems and time-aware modeling. Section 3 provides a detailed description of our proposed TimeWeaver model, including its key components: the time-aware augmentation mechanism and the dual-stream architecture. Section 4 presents our extensive experimental evaluation, including comparisons with state-of-the-art baselines, ablation studies, and hyperparameter analysis. Finally, we conclude the paper and outline potential future work in Section 5.

The main contributions of this paper are as follows:

- We propose a novel convolutional mechanism and a dual-stream network architecture to effectively model the diverse evolution of user behaviors across long- and short-term timescales. This architecture enables the model to retain long-term historical patterns while precisely capturing recent changes in user interests.
- To address the issue of insufficient temporal sensitivity, we introduce a positional encoding calibration method based on temporal features. This approach adjusts the global temporal offset of positional encodings via a learnable scaling factor, which embeds the temporal distribution properties of the data into the sequence representation. This significantly enhances the model's sensitivity to temporal dynamics.
- We have developed a prototype tool, which is available at https://github.com/AraVio/TimeWeaver (accessed on 27 September 2025). Extensive experiments on three public datasets demonstrate that TimeWeaver outperforms existing state-of-the-art methods across various evaluation metrics. Furthermore, through ablation studies and visualization analyses, we verify the effectiveness of our proposed dual-stream architecture, modern convolutional structure, and time-aware augmentation mechanism.

2. Related Work

2.1. Sequential Recommender System

A Sequential Recommender System aims to predict a user's future interests by capturing the sequential patterns in their historical interactions. Early approaches primarily relied on Markov chains [2,12,13], which model first-order or higher-order transition probabilities to capture dependencies. However, these methods have significant limitations in modeling sparse data and long-term dependencies. The rise of deep learning marked a fundamental paradigm shift in sequential recommender systems, as neural networks can learn high-level, abstract representations of dynamic user interests from complex interaction sequences.

RNNs and their variants, such as GRUs and LSTMs, were widely adopted first in this field due to their architectural alignment with sequential data. A landmark contribution is GRU4Rec [3], which first applied GRUs to model user behavior sequences and effectively captured the evolution of user interests. Subsequently, researchers proposed numerous enhancements to the RNN architecture, such as incorporating attention mechanisms to weigh the importance of different historical items. Around the same time, the Transformer model, based on self-attention, revolutionized the field of sequence modeling. The pioneering work SASRec [14] employed a unidirectional self-attention mechanism to efficiently capture long-range dependencies in parallel. Following this, BERT4Rec [15]

drew inspiration from the masked language model in natural language processing, which uses a bidirectional Transformer to learn deep contextual relationships between items. The success of these models has established self-attention as a core technology in sequential recommender systems.

Furthermore, researchers have explored using the strong local feature extraction capabilities of CNNs to model sequential patterns. Caser [4] stands as a pioneering work in this area. It innovatively represents a user's recent interaction sequence as an "image" and applies both horizontal and vertical convolutional kernels to capture sequential patterns and latent item associations, respectively. To overcome the receptive field limitations of traditional convolutions for long sequence modeling, NextItNet [6] drew inspiration from the TCN architecture. It introduces stacked causal dilated convolutions, which exponentially expand the model's receptive field without increasing computational cost, thereby enabling the capture of both long- and short-term dependencies. More recent research has trended towards fusing CNNs with other architectures to leverage their complementary strengths. For example, DACNN [16] constructs dual user-item interaction sequences and models them with an attention mechanism to capture richer dynamic information. In another example, AdaMCT [17] designs parallel CNN and Transformer branches, which use an adaptive gating mechanism to dynamically fuse local patterns from the CNN with global dependencies captured by the Transformer, striking a balance between performance and efficiency.

More recently, the landscape of sequential recommendation has been significantly influenced by the advent of Large Language Models (LLMs). This emerging trend generally involves two main approaches: using LLMs to generate auxiliary features for traditional models, or employing LLMs directly as the recommendation engine [18]. In the latter approach, user interaction histories are often formatted as natural language prompts to predict the next item. While LLMs offer powerful reasoning capabilities, they face challenges in efficiently processing long and complex user sequences, often requiring sophisticated retrieval-augmented techniques to select the most relevant historical interactions [19].

2.2. Time Aware for Sequential Recommender System

Temporal information is a critical factor in Sequential Recommender Systems, as temporal context significantly influences user interests and behavior patterns. Early sequential recommender systems models focused primarily on item order while ignoring the specific timestamps of interactions. This simplification limited the models' ability to comprehend the dynamic nature of user preferences. With advances in the field, researchers recognized the importance of the temporal dimension and began exploring effective ways to integrate it into model architectures.

Initially, research on time-aware sequential recommender systems centered on modeling time intervals. This involved introducing the time gap between adjacent user interactions as an additional feature, which was processed through simple linear transformations or embedding layers. However, this direct feature-concatenation approach often fails to capture the deeper, more complex interactions between temporal information and user interests. Following the application of the Transformer architecture in sequential recommender systems, researchers began to incorporate time into the self-attention mechanism. A representative work, TiSASRec [20], integrates time intervals into the attention weight calculation by constructing and encoding a time-difference matrix. While this approach improved the model's temporal awareness, it also substantially increased its complexity and computational cost.

Subsequent research has further diversified the approaches for time-aware modeling. For instance, MEANTIME [21] employs a multi-head attention mechanism to capture

varied temporal patterns within behavior sequences, while TimelyRec [22] adopts a hierarchical strategy to model user behavior at different time granularities. TAT4SRec [23] utilizes an encoder-decoder architecture that separately models timestamps and interacted items, integrating this information during the decoding stage to generate time-aware recommendations. Other works have focused on creating dynamic item representations; for example, DIDN [24] introduces a dynamic intent-aware module to construct evolving item embeddings by incorporating temporal order. Another novel direction explores the distribution of time intervals themselves, where [25] proposes data augmentation techniques to transform sequences with irregular time gaps into more uniform ones, thereby improving model performance. Although these methods have advanced the models' capabilities for temporal awareness, most are still constrained by complex attention mechanism designs and high computational costs.

In summary, while these studies have achieved significant progress in sequential recommendation, several key limitations persist. First, Transformer-based models, despite their power, are computationally expensive. Their self-attention mechanism applies a uniform approach to all historical items, making it difficult to distinguish between long-term stable preferences and short-term interest drifts. This challenge is further exacerbated in recent Large Language Model-based approaches. Despite their advanced reasoning capabilities, these models are often limited by the efficiency and scalability required to process long user histories. Second, most CNN-based architectures, including modern TCNs, lack a dedicated mechanism for explicitly modeling dependencies across different time scales. They typically apply a single convolutional structure to the entire sequence, which is insufficient for capturing the complex interplay between short-term and long-term user interests. Furthermore, existing time-aware methods often integrate temporal information through complex modifications to the attention mechanism or simple feature concatenation. These approaches can either increase model complexity or fail to fully capture the nuanced influence of time intervals on user behavior.

Our proposed TimeWeaver model is designed to directly address these shortcomings. It introduces a dual-stream architecture to overcome the limitations of single-paradigm models. Moreover, our novel Time-Aware Augmentation mechanism performs a dynamic calibration of positional encodings. By learning from the global temporal distribution properties within the data, this mechanism enriches the sequence representation with nuanced temporal dynamics without fundamentally altering the downstream network architecture.

3. Method

3.1. Problem Statement

In a Sequential Recommender System, we define a set of users $U = \left\{u_1, u_2, \ldots, u_{|U|}\right\}$ and a set of all items $V = \left\{v_1, v_2, \ldots, v_{|V|}\right\}$.

The historical interactions for each user $u \in U$ in the system are recorded as a chronologically ordered sequence. This sequence consists of a sequence of items $S_u = \left[v_1^u, v_2^u, \ldots, v_{n_u}^u\right]$ and a corresponding sequence of timestamps $T_u = \left[t_1^u, t_2^u, \ldots, t_{n_u}^u\right]$.

Here, $v_i^u \in V$ is the i-th item in user u ′s interaction history, and t_i^u is the precise time of that interaction, satisfying $t_1^u < t_2^u < \cdots < t_{n_u}^u$. The length of user u ′s interaction history is denoted by n_u.

The objective of time-aware sequential recommendation is to predict the item a user is most likely to interact with at the next timestep, given the user's historical item sequence S_u and timestamp sequence T_u. This task can be formally formulated as learning a conditional probability distribution:

$$p\left(v_{n_u+1}^u = v \middle| S_u, T_u\right) \tag{1}$$

where $v \in V$ is any candidate item from the entire item set.

3.2. Model Overview

To address the limitations of existing methods in modeling multi-scale temporal dependencies, we propose the TimeWeaver model, which is illustrated in Figure 1a. The model's overall workflow comprises three main stages: Time-Aware Augmentation, Dual-Stream Encoding, and Final Prediction.

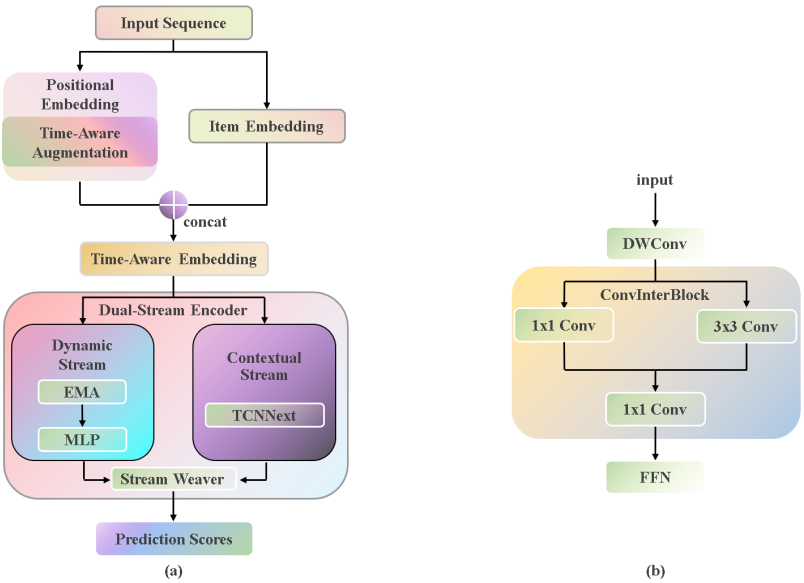


Figure 1. The overall architecture of TimeWeaver. (**a**) The main architecture of the model. An input sequence is first processed by the Time-Aware Augmentation module to extract time-aware embeddings. These embeddings are then processed by the dual-stream encoder, consisting of a context stream and a dynamic stream, and their outputs are fused to generate the final prediction scores. (**b**) The structure of the TCNNext module. As the core component of the context stream, TCNNext consists of a depthwise separable convolutional layer, a parallel convolutional interaction module, and a feed-forward neural network.

First, the model processes the input item sequence S_u and timestamp sequence T_u in an embedding and augmentation stage. This stage fuses the item embeddings with temporally-adjusted positional embeddings produced by our proposed Time-Aware Augmentation mechanism. This mechanism adjusts positional information by leveraging the time intervals between interactions, which generates an initial sequence representation $H^{(0)}$ that is sensitive to temporal dynamics. This representation then serves as the input to the subsequent encoding modules.

Next, $H^{(0)}$ is fed into a dual-stream encoder, which consists of a stack of N identical modules. In each encoding layer, the hidden state $H^{(l-1)}$ from the previous layer is processed in parallel by a context stream and a dynamic stream. The context stream employs a modified TCN structure to better capture the user's long-term preferences, while the dynamic stream leverages an EMA mechanism to sensitively capture rapidly changing short-term interests. The outputs from these two streams are then fused by a Stream Weaver and combined with a residual connection to produce the updated layer representation $H^{(l)}$.

After passing through all encoding layers, the model uses the last vector from the final sequence representation $H^{(N)}$ to compute an inner product with the item embedding matrix. This procedure yields the final predicted probability distribution for the next item.

3.3. Time-Aware Augmentation

Given a user's item sequence $S_u = \{v_1^u, v_2^u, \ldots, v_{n_u}^u\}$ and its corresponding timestamp sequence $T_u = \{t_1, t_2, \ldots, t_n\}$, the model first converts each item v_i^u into a d-dimensional vector representation e_i^{item} using an item embedding matrix $E^{item} \in R^{|\mathbb{I}| \times d}$. To incorporate sequential information, we also employ a standard positional embedding matrix $E^{pos} \in R^{L_{max} \times d}$ to generate an encoding p_i for each position i. Here, L_{max} is the maximum sequence length supported by the model, and sequences exceeding this length are truncated.

The core of this mechanism is the dynamic calibration of positional encodings using temporal information. First, we discretize each raw timestamp t_i into an integer index within the range $[0, T_{max} - 1]$ by applying a modulo operation:

$$\widetilde{t_i} = t_i \; mod \; T_{max} \tag{2}$$

where T_{max} is the upper bound for the time index. Subsequently, a time embedding layer maps these discrete time indices $\widetilde{t_i}$ to vector representations, yielding $e_i^{time} = E^{time}\left[\widetilde{t_i}\right] \in R^d$.

We then introduce a global position offset strategy, which computes the mean of the time embeddings at each position i across all sequences within a batch. For a batch $b \in [1, B]$, let $e_{b,i}^{time}$ denote the time embedding at position i of the b-th sequence. The batch-level average time embedding for position i is then:

$$\bar{e}_i^{time} = \frac{1}{B} \sum_{b=1}^{B} e_{b,i}^{time} \in R^d \tag{3}$$

This "global position offset" strategy is central to our time-aware augmentation. The resulting vector $\bar{e}_i^{time}$ does not represent the temporal information of a single sequence; rather, it captures a shared temporal pattern across all sequences in the batch at a specific position i. For instance, if user interactions at the beginning of sessions typically exhibit long time intervals, the average time embedding will encode this pattern. Conversely, if interactions toward the end of sequences are usually rapid, the corresponding average vector will reflect this higher frequency of interaction. We then scale this mean vector by a learnable scalar parameter β to adjust the original positional encoding:

$$\hat{p}_i = p_i + \beta \cdot \bar{e}_i^{time} \tag{4}$$

The parameter β is learned adaptively during training to dynamically regulate the influence of temporal information on the positional encodings.

Finally, the temporally-augmented sequence representation is formed by fusing the item embeddings with the calibrated positional encodings. By adding this shared temporal pattern back to the original positional encoding, we are effectively calibrating it. The standard positional encoding p_i only conveys order. Our calibrated encoding $\hat{p}_i$, however, conveys richer, time-aware information: "this is the i-th item, and it is typically associated with this specific temporal pattern." This allows the model to better distinguish between positions based not only on their order but also on the behavioral patterns associated with that order. To stabilize the training, we apply Layer Normalization and Dropout:

$$E^{(0)} = \text{Dropout}\Big(\text{LayerNorm}\Big(\Big\{e_i^{item} + \hat{p}_i\Big\}_{i=1}^{n}\Big)\Big) \tag{5}$$

Here, $E^{(0)} \in R^{n \times d}$ is the resulting initial sequence representation, which serves as the input to the dual-stream encoder. In addition to preserving order, a key aspect of this mechanism is the effective integration of the data's global temporal distribution properties into the sequence representation.

3.4. Dual-Stream Encoder

The dual-stream encoder consists of L stacked identical layers, taking the time-aware augmented sequence representation $E^{(0)}$ as its initial input. Within each encoding layer $l \in [1, L]$, the output from the preceding layer $H^{(l-1)}$ (where $H^{(0)} = E^{(0)}$) is fed in parallel into two specialized streams: a context stream and a dynamic stream. The outputs of these two streams are then fused and integrated with a residual connection, which produces the layer's final output $H^{(l)}$.

3.4.1. Context Stream

The context stream is designed to efficiently capture medium- and long-term contextual information within user behavior sequences. Its core is a modern temporal convolution module, which we term TCNNext. The design of this module is inspired by "modern convolution," as it adopts the block-like structure of Transformers. It also employs large-kernel convolution to achieve a vast effective receptive field, enhancing its ability to model long-term dependencies, as illustrated in Figure 2. The TCNNext module leverages recent advances in modern convolutional network design and is optimized for sequential recommendation tasks.

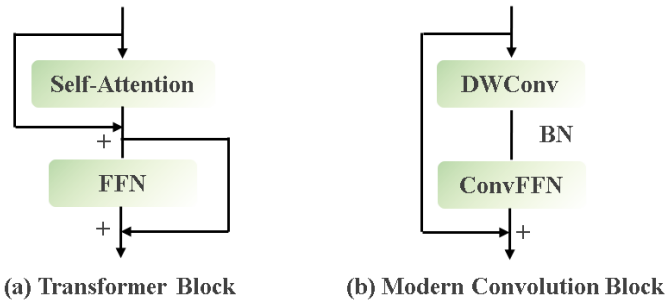


Figure 2. A comparison of the architectures of (**a**) a standard Transformer block and (**b**) a modern convolution block. The modern convolution block adapts the Transformer architecture, primarily by replacing the Self-Attention module with DWConv and substituting the feed-forward network with a convolutional version (ConvFFN). The TCNNext module in this paper is designed based on this modern convolutional block.

For the l-th layer in the encoder, the context stream takes the output from the previous layer, $H^{(l-1)}$, as its input. It then refines dynamic patterns in the sequence through a series of specialized operations. As shown in Figure 1b, the TCNNext module consists of three key sub-layers connected in series. Each sub-layer is followed by a residual connection and layer normalization to ensure stable information flow and training convergence.

The first sub-layer of the module is a depthwise separable convolution. Unlike standard convolution, this operation independently applies a size-adaptive large-kernel depthwise convolution to each feature dimension (see Section 4.4 for details). This approach effectively expands the receptive field at a low computational cost, allowing it to capture longer-range dependencies. This convolution is immediately followed by a batch normalization layer. Specifically, the input $X = H^{(l-1)}$ first passes through this convolutional layer. A residual connection to the original input is then added, and the result is passed through layer normalization to yield the intermediate representation X':

$$X' = \text{LayerNorm}(\text{DWConv}(X) + X) \tag{6}$$

Here, $\text{DWConv}(\cdot)$ denotes the combination of large-kernel depthwise separable convolution and batch normalization.

The second sub-layer is a parallel convolutional interaction module designed for the complex integration of temporal features. As formalized in Equations (7)–(9), the input

X' is processed by two parallel convolutional branches. Their outputs are subsequently combined using a gating mechanism, which facilitates a dynamic and non-linear fusion of local contextual information captured from different receptive fields.

$$Z_1 = \text{Conv1}D_{1\times1}(X'), \quad Z_2 = \text{Conv1}D_{3\times3}(X') \tag{7}$$

$$G = (Z_1 \odot \sigma(Z_2)) + (Z_2 \odot \sigma(Z_1)) \tag{8}$$

$$X'' = \text{LayerNorm}\big(\text{Conv1}D'_{1\times1}(G) + X'\big) \tag{9}$$

where σ denotes the GELU activation function and $\odot$ represents element-wise multiplication.

The final sub-layer is a standard position-wise feed-forward network (FFN), identical to the FFN architecture used in Transformers. It consists of two linear transformation layers with an intermediate GELU activation function. This FFN applies an independent non-linear transformation to the representation at each time step, thereby enhancing the model's representational power. A residual connection is added between the input X'' and the FFN's output, followed by layer normalization. This yields the final output of the context stream for the current encoding layer, $H_C^{(l)}$:

$$H_C^{(l)} = \text{LayerNorm}\big(\text{FFN}\big(X''\big) + X''\big) \tag{10}$$

By stacking these three sub-layers, the context stream comprehensively extracts rich dynamic interest features from the user sequence, spanning from local to medium-term dependencies.

3.4.2. Dynamic Stream

Unlike the context stream, which focuses on modeling long-term dependencies, the dynamic stream is designed to accurately capture immediate fluctuations in user interest. Its core design principle is a high sensitivity to recent user behavior. A user's immediate needs are often dominated by their most recent interactions; therefore, the model must assign greater weight to these recent signals.

At the l-th layer of the encoder, the stream receives the output from the preceding layer, $H^{(l-1)}$, and first processes it using an EMA. This mechanism aims to suppress noise arising from short-term interactions by recursively smoothing sequence features, thereby reinforcing and preserving persistent signals that span multiple interaction clusters. The traditional recursive formulation of EMA is:

$$s_t = \gamma \cdot x_t + (1-\gamma) \cdot s_{t-1} \tag{11}$$

where s_t is the smoothed representation at timestep t and $\gamma \in (0,1)$ is the smoothing factor. However, this recurrent definition is not amenable to efficient implementation on modern parallel computing architectures.

Therefore, we adopt a non-recurrent, vectorized computation scheme that is mathematically equivalent to the recursive form but enables full parallelization. For an input sequence representation $X \in R^{B\times T\times d}$, where B is the batch size, T is the sequence length, and d is the feature dimension, the vector s_t at each timestep t in the EMA-smoothed sequence S is computed via a normalized, weighted cumulative sum.

We define a vector of decay exponents $p = [T-1, T-2, \ldots, 0]$ and use it to generate two key weight vectors. The first is a normalization weight vector, w', where each element is calculated as $w'_j = (1-\alpha)^{T-1-j}$. The second is a primary weight vector, w, which is derived from w'. Its first element is identical to the first element of w', while all subsequent elements are their counterparts in w' multiplied by a decay factor α.

The smoothed sequence S is obtained by first computing a weighted cumulative sum C of the input sequence X using the weight vector w. The resulting sequence is subsequently normalized element-wise by the vector w', as formalized in Equations (12) and (13):

$$C = \text{cumsum}(X \odot w) \tag{12}$$

$$S = C \varnothing w\prime \tag{13}$$

where $\text{cumsum}(\cdot)$ represents the prefix sum operation along the temporal dimension, and $\odot$ and $\varnothing$ denote element-wise multiplication and division, respectively.

To further enhance the representational power of the dynamic stream, the EMA-smoothed features are passed through a linear projection layer. This projection aligns the features with the space learned by the context stream. Thus, the output of the dynamic stream at layer l, denoted as $H_T^{(l)}$, is formulated as:

$$H_T^{(l)} = \text{EMA}\left(H^{(l-1)}\right) W_T^{(l)} + b_T^{(l)} \tag{14}$$

Here, $\text{EMA}(\cdot)$ denotes the parallelized exponential moving average operation. The terms $W_T^{(l)}$ and $b_T^{(l)}$ are the weight matrix and bias vector, respectively, of the linear layer in the dynamic stream at layer l.

3.4.3. Stream Weaver

To synergistically integrate the complementary information extracted by the two parallel streams, we introduce a core fusion module at the end of each encoding layer, which we term the "Stream Weaver". This module is designed to effectively aggregate feature representations from the two different temporal scales.

Specifically, for the output of layer l, the representation from the context stream, $H_C^{(l)}$, and the dynamic stream, $H_T^{(l)}$, are first concatenated along the feature dimension. This combined representation is then fed into a linear layer. This layer utilizes learnable parameters to adaptively weight the features from each stream and projects the fused information back to the original hidden dimension. Finally, we incorporate a residual connection from the previous layer's input, $H^{(l-1)}$, followed by layer normalization. This entire process is formulated as:

$$H^{(l)} = \text{LayerNorm}\left(\left[H_C^{(l)}\,;\, H_T^{(l)}\right] W_F^{(l)} + b_F^{(l)} + H^{(l-1)}\right) \tag{15}$$

where $[\cdot\,;\,\cdot]$ denotes the concatenation operation along the feature dimension. The terms $W_F^{(l)}$ and $b_F^{(l)}$ are the weight matrix and bias vector, respectively, for the fusion linear layer at layer l.

3.5. Prediction Layer

After passing through the L layers of the dual-stream encoder, the model yields the final hidden state representation of the sequence, $H^{(L)} \in R^{N_u \times d}$. To predict the user's next behavior, we take the vector corresponding to the last time step, denoted as $h_{N_u}^{(L)}$, which aggregates the user's dynamic long- and short-term interests. The preference score $\hat{y}_i$ for each candidate item i from the entire item set $\mathcal{V}$ is then calculated as the inner product of this final user representation and the item's embedding vector e_i. This prediction process is formalized as:

$$\hat{y}_i = \left(h_{N_u}^{(L)}\right)^{Te_i} \tag{16}$$

To optimize the model parameters, we employ the cross-entropy (CE) loss function as the training objective, following the standard paradigm for sequential recommendation. The task is framed as a multi-class classification problem: given a user's historical sequence, the model must correctly classify the next item of interaction from the entire item set $\mathcal{V}$. For any given training instance, the loss is calculated as:

$$\mathcal{L} = -\log\left(\frac{\exp(\hat{y}_g)}{\sum_{i\in\mathcal{V}}\exp(\hat{y}_i)}\right) \tag{17}$$

Here, $g \in \mathcal{V}$ is the ground-truth item that the user interacts with at the next time step. The denominator normalizes the prediction scores over all candidate items via the Softmax function. Minimizing this loss function trains the model to learn sequence representations that accurately predict a user's future interests.

4. Experiments

4.1. Training Configuration

All experiments were conducted on a hardware platform equipped with a 12-core Intel(R) Xeon(R) Silver 4214R CPU @ 2.40 GHz and an NVIDIA RTX 3080 Ti GPU with 12 GB of VRAM. The software environment consisted of Ubuntu 20.04, and all models were implemented in Python 3.8 using the PyTorch 1.11.0 framework, with GPU acceleration provided by CUDA 11.6.

4.2. Datasets

Our experimental evaluation is conducted on three widely-used Amazon review datasets: Beauty, Sports, and Toys (The datasets are publicly available at http://jmcauley.ucsd.edu/data/amazon/, accessed on 27 September 2025). These datasets are selected because they are derived from real-world consumer scenarios and exhibit significant differences in item categories, user behavior patterns, and data density. This diversity allows for a robust evaluation of our model's generalizability across various user preferences and item distributions.

To adapt the data for the sequential recommendation task, we applied a consistent preprocessing pipeline. First, we convert all user behaviors, such as ratings and reviews, into binary implicit feedback (i.e., an "interaction"). Second, we generate a chronological sequence of interactions for each user based on the timestamps. Finally, to mitigate noise from data sparsity, we filter the data by retaining only users and items with at least five interactions. The statistics of the resulting datasets, which are used for model training and testing, are summarized in Table 1.

Table 1. Statistics of the Datasets.

Dataset	# Users	# Items	# Avg. Length	# Actions	Sparsity
Beauty	22,363	12,101	8.9	198,502	99.93%
Sports	25,598	18,357	8.3	296,337	99.95%
Toys	19,412	11,924	8.6	167,597	99.93%

The symbol "#" denotes "The number of".

4.3. Evaluation Metrics

For evaluation, we adopt the standard leave-one-out strategy common in sequential recommendation. Specifically, for each user's interaction sequence, the final item is used as the ground truth for the test set, the second-to-last item is used for validation, and all preceding items are used for training. To ensure a rigorous evaluation, we rank the target

item against the entire item corpus, rather than employing negative sampling techniques that might introduce bias.

We evaluate model performance using two standard metrics: Hit Rate (HR@k) and Normalized Discounted Cumulative Gain (NDCG@k). HR@k measures the fraction of times the ground-truth item appears in the top-k recommended list, which is equivalent to Recall@k under this setting. NDCG@k is a position-aware metric that evaluates ranking quality by assigning greater importance to items ranked higher. We will report the performance for k values of 5, 10, and 20. For both metrics, higher values indicate better recommendation performance.

4.4. Baselines & Implementation Details

To facilitate a comprehensive evaluation, we compare our proposed model against a set of representative baseline methods that span from classic to state-of-the-art architectures:

- CNN-based Methods: We include Caser [4], which utilizes convolutional operations to extract high-order Markov patterns from user sequences.
- RNN-based Methods: We select GRU4Rec [3], a pioneering model in this domain that uses GRUs to effectively capture temporal dependencies in user behavior sequences.
- Transformer-based Methods: This category represents the current mainstream in sequential recommendation. We select three prominent models: SASRec [14] employs a unidirectional self-attention mechanism to model user sequences; BERT4Rec [15] uses a bidirectional self-attention mechanism and learns deeper representations via a MLM task; TiSASRec [20] extends SASRec by incorporating time interval information into the self-attention computation to more accurately model dynamic user interests.
- Emerging Architectures: We also include two recent models. FMLP-Rec [8] is a pure MLP-based architecture that uses a filter module to suppress noise. LRURec [26] is built upon Linear Recurrent Units, aiming to combine the inference efficiency of RNN-like models with the parallel training capabilities of Transformer-like models.

All experiments are conducted within a unified framework. For all baselines, we tune hyperparameters based on a combination of the recommendations from their original papers and a grid search [27,28]. We use the Adam optimizer with a learning rate of 0.001 and a batch size of 256. The embedding dimension is set to 64 and the dropout rate is set to 0.5 for all models. During data processing, the maximum sequence length is truncated to 50. All models are trained from scratch without any pre-trained parameters.

Furthermore, for our model, the kernel size of the depthwise separable convolution layer is adaptively configured based on the maximum sequence length of the dataset. This strategy balances the effective receptive field and computational efficiency across different datasets. Specifically, the base kernel size is set to one-third of the maximum sequence length. If this value is even, it is incremented by one to ensure it is odd. To prevent excessive parameter growth and computational cost on sequences with extreme lengths, we cap the maximum kernel size at 15.

4.5. Overall Performance Comparison

The experimental results in Table 2 provide a comprehensive performance comparison between our proposed model and various mainstream baselines. It is evident that traditional models like Caser and GRU4Rec consistently lag behind on all datasets. Specifically, Caser uses CNNs to extract local patterns but struggles to capture long-range dependencies in user sequences due to the limited receptive field of its convolutional operations. Similarly, GRU4Rec, which relies on GRUs for temporal modeling, performs reasonably well on short sequences but suffers from information loss when handling longer ones. This observation validates the limitations of traditional sequential models as discussed in our introduction.

Table 2. Recommendation performance comparison of all models on the three datasets. All metric values are reported to four decimal places. The best results are highlighted in bold, and the second-best are underlined. For both HR and NDCG, higher values indicate superior performance. The "Improv." column shows the percentage of relative improvement of TimeWeaver over the strongest baseline. * denotes a statistically significant improvement over the strongest baseline, determined by a paired *t*-test ($p < 0.05$).

Dataset	Metric	Caser	GRU4Rec	SASREC	BERT4Rec	TiSASRec	FMLPRec	LRURec	TimeWeaver	Improv.
Beauty	HR@5	0.0111	0.0158	0.0389	0.0437	0.0663	0.0406	0.0670	**0.0695**	3.73% *
	HR@10	0.0200	0.0276	0.0626	0.0670	0.0907	0.0615	0.0915	**0.0971**	6.12% *
	HR@20	0.0358	0.0449	0.0914	0.1005	0.1237	0.0909	0.1223	**0.1328**	7.36% *
	NDCG@5	0.0067	0.0096	0.0250	0.0277	0.0485	0.0268	0.0479	**0.0496**	2.27% *
	NDCG@10	0.0096	0.0134	0.0326	0.0352	0.0563	0.0335	0.0558	**0.0585**	3.91% *
	NDCG@20	0.0136	0.0178	0.0399	0.0436	0.0647	0.0409	0.0636	**0.0675**	4.33% *
Sports	HR@5	0.0092	0.0135	0.0175	0.0290	0.0335	0.0183	0.0374	**0.0405**	8.29% *
	HR@10	0.0165	0.0215	0.0289	0.0444	0.0479	0.0301	0.0524	**0.0576**	9.92% *
	HR@20	0.0263	0.0343	0.0448	0.0681	0.0683	0.0477	0.0742	**0.0831**	11.99% *
	NDCG@5	0.0057	0.0090	0.0116	0.0185	0.0237	0.0116	0.0260	**0.0281**	8.08% *
	NDCG@10	0.0081	0.0116	0.0153	0.0235	0.0283	0.0154	0.0308	**0.0336**	9.09% *
	NDCG@20	0.0105	0.0148	0.0193	0.0294	0.0334	0.0199	0.0363	**0.0400**	10.19% *
Toys	HR@5	0.0117	0.0150	0.0449	0.0501	0.0637	0.0487	0.0719	**0.0745**	3.62% *
	HR@10	0.0199	0.0266	0.0648	0.0731	0.0837	0.0693	0.0967	**0.1012**	4.65% *
	HR@20	0.0331	0.0437	0.0918	0.1061	0.1111	0.0956	0.1266	**0.1354**	6.95% *
	NDCG@5	0.0072	0.0090	0.0303	0.0337	0.0468	0.0329	0.0529	**0.0547**	3.40% *
	NDCG@10	0.0098	0.0127	0.0367	0.0411	0.0533	0.0396	0.0609	**0.0633**	3.94% *
	NDCG@20	0.0131	0.0169	0.0436	0.0494	0.0602	0.0462	0.0684	**0.0718**	4.97% *

In contrast, Transformer-based models, including SASRec, BERT4Rec, and TiSASRec, achieve significant performance gains. This demonstrates the advantage of the self-attention mechanism in capturing complex and long-range dependencies among items. Notably, TiSASRec, which incorporates time interval information, shows a distinct performance improvement over SASRec and BERT4Rec. This result underscores the importance of explicitly modeling temporal dynamics, which is a core motivation for our research.

Among the emerging architectures, LRURec demonstrates highly competitive performance, outperforming most mainstream baselines on the majority of metrics. We therefore consider it a primary baseline for comparison. This suggests that its linear recurrent structure, which combines the advantages of RNN-like and Transformer-like models, is effective for the sequential recommendation task. In contrast, while the pure MLP-based FMLP-Rec performs adequately on some metrics, its overall performance does not surpass the top-performing Transformer and LRURec models, indicating its limitations in finely capturing sequential dependencies.

Finally, our proposed TimeWeaver model achieves state-of-the-art performance, outperforming all baseline methods, including LRURec, across all evaluation metrics on all three datasets (Beauty, Sports, and Toys). Specifically, compared to the strongest baseline, LRURec, TimeWeaver delivers average relative improvements of 4.62%, 9.59%, and 4.59% across all metrics on the Beauty, Sports, and Toys datasets, respectively. This consistent and significant outperformance strongly validates the synergy among our proposed dual-stream architecture, the time-aware augmentation mechanism, and the modern TCN module.

4.6. Ablation Study

To investigate the individual contribution of each core component of TimeWeaver, we conduct a systematic ablation study on the Beauty, Sports, and Toys datasets. By systematically removing or replacing key components, we create several model variants and measure the resulting performance changes to quantify the effectiveness of our design.

All ablation variants are trained with the identical hyperparameter configuration as the full model to ensure a fair comparison. We designed the following four ablation variants:

- *w/o* Time: This variant removes the time-aware augmentation module and uses only standard static positional encodings, thus not explicitly modeling the time intervals between interactions. It is designed to evaluate the contribution of explicit temporal information to sequence modeling and interest perception.
- *w/o* Dynamic Stream: This variant ablates the dynamic stream, relying solely on the context stream for sequence modeling. It is used to assess the impact of the EMA mechanism on capturing short-term interest fluctuations.
- *w/o* Context Stream: We remove the context stream and retain only the EMA-based dynamic stream to evaluate the contribution of long-term interest modeling to the overall performance.
- *w/o* ConvInter: This variant removes the parallel convolutional interaction module from the TCNNext block, retaining only the main convolution and feed-forward structures. It is used to evaluate the effectiveness of the complex local temporal feature fusion mechanism.

The performance of each ablation variant is presented in Table 3. As a general observation, the removal of any single key component leads to a significant degradation in model performance. A detailed analysis is as follows:

Table 3. Results of the ablation study. "Default" denotes the full TimeWeaver model.

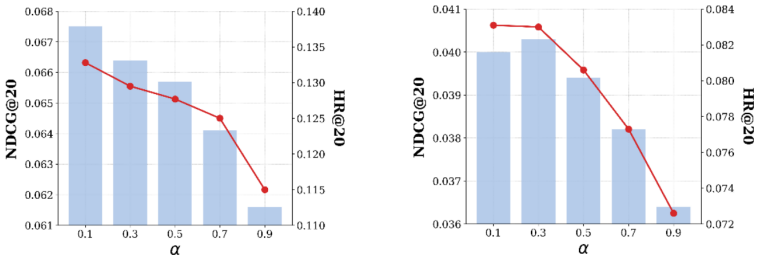
Methods	Beauty		Sports		Toys	
	HR@20	NDCG@20	HR@20	NDCG@20	HR@20	NDCG@20
Default	0.1328	0.0675	0.0831	0.0400	0.1354	0.0718
w/o Time	0.1261	0.0646	0.0806	0.0394	0.1299	0.0687
w/o Dynamic Stream	0.1294	0.0646	0.0792	0.0384	0.1313	0.0692
w/o Context Stream	0.1288	0.0659	0.0794	0.0384	0.1308	0.0692
w/o ConvInter	0.1275	0.0650	0.0788	0.0379	0.1329	0.0705

- Time-Aware Augmentation: On average, removing this mechanism causes a performance drop of 4.04% in HR@20 and 3.37% in NDCG@20 across all datasets. This indicates that static positional encodings alone are insufficient to capture the temporal dynamics of user behavior, and that explicitly modeling temporal features effectively enhances the model's perception of interest evolution.
- Dual-Stream Architecture: Ablating either the dynamic or the context stream results in a clear performance decline across all metrics. This demonstrates that the two streams are complementary in capturing both long-term stable preferences and short-term immediate needs, making the dual-stream design essential for modeling multi-scale interest dynamics.
- Parallel Convolutional Interaction: Removing this module leads to a consistent performance drop across all metrics, which highlights its important role in dynamic feature fusion and the extraction of complex temporal patterns.

These results demonstrate that TimeWeaver's innovative components are critical for synergistically modeling multi-scale temporal features. They contribute significantly to the model's superior recommendation performance and validate the effectiveness and necessity of our architectural design.

4.7. Hyperparameter Sensitivity Analysis

In this section, we investigate the impact of TimeWeaver's key hyperparameters on its performance to validate its robustness and guide hyperparameter selection. Specifically, we analyze the sensitivity to two key parameters: the EMA decay rate α, which has a significant impact on the dual-stream architecture, and the time scaling factor β. Figure 3 illustrates the performance changes in NDCG@20 and HR@20 on the Beauty and Sports datasets as α varies.

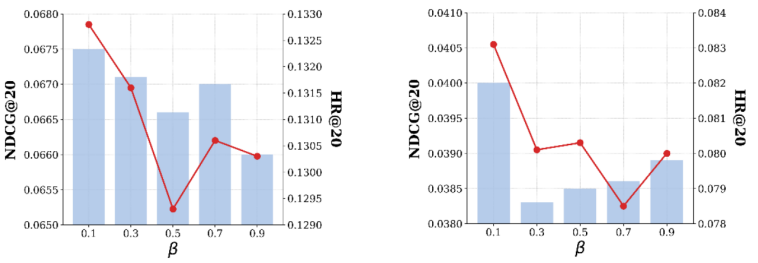


(a) Beauty (b) sports

Figure 3. Sensitivity to the EMA Decay Coefficient. This parameter governs how rapidly the dynamic stream discounts historical information. The blue bars represent the NDCG@20 scores (left y-axis), and the red line represents the HR@20 scores (right y-axis).

The results show that as the EMA decay rate α increases, model performance exhibits a downward trend on both datasets. This phenomenon suggests that a smaller decay rate is more effective for modeling the dynamic stream. Although this stream is designed to capture short-term interests, it must retain sufficient historical context to maintain the coherence of the sequence. Conversely, an excessively high α value causes the model to overly emphasize the most recent interactions, thereby ignoring the broader session context and impairing recommendation accuracy.

Figure 4 presents the model's sensitivity to the time scaling factor, which controls the magnitude of the temporal adjustment applied to the positional encodings. The results show that performance peaks when the scaling factor is relatively small. This indicates that a modest scaling factor provides a more effective temporal calibration for the static positional encodings. In contrast, an overly large factor can allow the temporal signal to dominate the original positional information, which negatively affects model performance.



(a) Beauty (b) sports

Figure 4. Sensitivity to the Time Factor. This parameter regulates the magnitude of the temporal calibration applied to the positional representations. The blue bars represent the NDCG@20 scores (left y-axis), and the red line represents the HR@20 scores (right y-axis).

Overall, TimeWeaver demonstrates good robustness to both key hyperparameters, with its performance remaining relatively stable across a reasonable range of their values.

5. Conclusions

In this paper, we proposed TimeWeaver, a time-aware dual-stream network designed to efficiently model temporal dependencies and dynamic changes within user behavior sequences. By leveraging large-kernel convolution, a time-aware augmentation mechanism, and a dual-stream architecture, TimeWeaver effectively captures users' long-term preferences while remaining highly sensitive to recent changes in their interests. A key contribution of this work is demonstrating that a specialized, dual-stream architecture can effectively resolve the inherent trade-off between capturing long-range dependencies and responding to short-term interest shifts. Experimental results demonstrate that TimeWeaver outperforms existing state-of-the-art models on several public datasets, which validates its superiority in modeling dependencies across multiple time scales. Furthermore, the ablation study and hyperparameter analysis confirmed the effectiveness and significant contribution of each innovative module to the model's performance.

Despite these promising results, our work has limitations that open avenues for future research. One limitation of the current study is that TimeWeaver, similar to many state-of-the-art sequential recommenders, operates in a transductive setting. Consequently, the model can only recommend items that were present in the training set and cannot inherently handle new items introduced after training, a classic manifestation of the "new item cold-start" problem. Future work could address this challenge by extending TimeWeaver into an inductive framework. A promising direction involves incorporating item side-information, such as textual descriptions or visual attributes. By training a content encoder (e.g., a pre-trained language model) alongside the main model, TimeWeaver could learn to dynamically generate embeddings for new items from their features. This would enable the model to recommend items outside its original training vocabulary, significantly enhancing its practical applicability in dynamic environments like e-commerce, where new products are constantly introduced. In addition to addressing this primary limitation, future work could also involve exploring more complex inter-stream interaction mechanisms or extending this time-aware framework to other recommendation scenarios rich in temporal dynamics.

Author Contributions: Conceptualization, Y.L.; methodology, Y.L. and Y.M.; investigation, T.W.; data curation, T.W.; writing—original draft preparation, T.W.; writing—review and editing, Y.L.; supervision, Y.L.; funding acquisition, Y.L. and Y.M. All authors have read and agreed to the published version of the manuscript.

Funding: This work is funded by The National Social Science Fund of China under Grant No. 24BGL111.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: This study utilized publicly available datasets. No new datasets were generated for this research. The datasets can be found at: http://jmcauley.ucsd.edu/data/amazon/ (accessed on 27 September 2025).

Conflicts of Interest: The authors have no conflict of interest.

References

1. Boka, T.F.; Niu, Z.; Neupane, R.B. A survey of sequential recommendation systems: Techniques, evaluation, and future directions. *Inf. Syst.* **2024**, *125*, 102427. [CrossRef]
2. He, R.; McAuley, J. Fusing similarity models with markov chains for sparse sequential recommendation. In Proceedings of the 2016 IEEE 16th International Conference on Data Mining (ICDM), Barcelona, Spain, 12–15 December 2016; IEEE: New York, NY, USA; pp. 191–200.

3. Hidasi, B.; Karatzoglou, A.; Baltrunas, L.; Tikk, D. Session-based recommendations with recurrent neural networks. In Proceedings of the 4th International Conference on Learning Representations (ICLR), San Juan, Puerto Rico, 2–4 May 2016.
4. Tang, J.; Wang, K. Personalized top-n sequential recommendation via convolutional sequence embedding. In Proceedings of the Eleventh ACM International Conference on Web Search and Data Mining, Los Angeles, CA, USA, 5–9 February 2018; pp. 565–573.
5. Farha, Y.A.; Gall, J. Ms-tcn: Multi-stage temporal convolutional network for action segmentation. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, Long Beach, CA, USA, 15–20 June 2019; pp. 3575–3584.
6. Yuan, F.; Karatzoglou, A.; Arapakis, I.; Jose, J.M.; He, X. A simple convolutional generative network for next item recommendation. In Proceedings of the Twelfth ACM International Conference on Web Search and Data Mining, Melbourne, Australia, 11–15 February 2019; pp. 582–590.
7. Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A.N.; Kaiser, L.; Polosukhin, I. Attention is all you need. *Adv. Neural Inf. Process. Syst.* **2017**, *30*, 5998–6008.
8. Zhou, K.; Yu, H.; Zhao, W.X.; Wen, J.R. Filter-enhanced MLP is all you need for sequential recommendation. In Proceedings of the ACM Web Conference 2022, Lyon, France, 25–29 April 2022; pp. 2388–2399.
9. Lara-Benítez, P.; Carranza-García, M.; Luna-Romera, J.M.; Riquelme, J.C. Temporal convolutional networks applied to energy-related time series forecasting. *Appl. Sci.* **2020**, *10*, 2322. [CrossRef]
10. Luo, D.; Wang, X. Moderntcn: A modern pure convolution structure for general time series analysis. In Proceedings of the Twelfth International Conference on Learning Representations, Vienna, Austria, 7–11 May 2024; pp. 1–43.
11. Pitka, T.; Bucko, J.; Krajči, S.; Krídlo, O.; Guniš, J.; Šnajder, Ľ.; Antoni, Ľ.; Eliaš, P. Time analysis of online consumer behavior by decision trees, GUHA association rules, and formal concept analysis. *J. Mark. Anal.* **2025**, *13*, 29–52. [CrossRef]
12. He, R.; Kang, W.C.; McAuley, J. Translation-based recommendation. In Proceedings of the Eleventh ACM Conference on Recommender Systems, Como, Italy, 27–31 August 2017; pp. 161–169.
13. Rendle, S.; Freudenthaler, C.; Schmidt-Thieme, L. Factorizing personalized markov chains for next-basket recommendation. In Proceedings of the 19th International Conference on World Wide Web, Raleigh, NC, USA, 26–30 April 2010; pp. 811–820.
14. Kang, W.C.; McAuley, J. Self-attentive sequential recommendation. In Proceedings of the 2018 IEEE International Conference on Data Mining (ICDM), Singapore, 17–20 November 2018; IEEE: New York, NY, USA, 2018; pp. 197–206.
15. Sun, F.; Liu, J.; Wu, J.; Pei, C.; Lin, X.; Ou, W.; Jiang, P. BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer. In Proceedings of the 28th ACM International Conference on Information and Knowledge management, Beijing, China, 3–7 November 2019; pp. 1441–1450.
16. Chen, Q.; Li, G.; Zhou, Q.; Shi, S.; Zou, D. Double attention convolutional neural network for sequential recommendation. *ACM Trans. Web* **2022**, *16*, 1–23. [CrossRef]
17. Jiang, J.; Zhang, P.; Luo, Y.; Li, C.; Kim, J.B.; Zhang, K.; Wang, S.; Xie, X.; Kim, S. AdaMCT: Adaptive mixture of CNN-transformer for sequential recommendation. In Proceedings of the 32nd ACM International Conference on Information and Knowledge Management, Birmingham, UK, 21–25 October 2023; pp. 976–986.
18. Wu, L.; Zheng, Z.; Qiu, Z.; Wang, H.; Gu, H.; Shen, T.; Qin, C.; Zhu, C.; Zhu, H.; Liu, Q.; et al. A survey on large language models for recommendation. *World Wide Web* **2024**, *27*, 60. [CrossRef]
19. Lin, J.; Shan, R.; Zhu, C.; Du, K.; Chen, B.; Quan, S.; Tang, R.; Yu, Y.; Zhang, W. Rella: Retrieval-enhanced large language models for lifelong sequential behavior comprehension in recommendation. In Proceedings of the ACM Web Conference 2024, Singapore, 13–17 May 2024; pp. 3497–3508.
20. Li, J.; Wang, Y.; McAuley, J. Time interval aware self-attention for sequential recommendation. In Proceedings of the 13th International Conference on Web Search and Data Mining, Houston, TX, USA, 3–7 February 2020; pp. 322–330.
21. Cho, S.M.; Park, E.; Yoo, S. MEANTIME: Mixture of attention mechanisms with multi-temporal embeddings for sequential recommendation. In Proceedings of the 14th ACM Conference on Recommender Systems, Virtual, 22 September 2020; pp. 515–520.
22. Cho, J.; Hyun, D.; Kang, S.; Yu, H. Learning heterogeneous temporal patterns of user preference for timely recommendation. In Proceedings of the Web Conference, Virtual, 12–23 April 2021; pp. 1274–1283.
23. Zhang, Y.; Yang, B.; Liu, H.; Li, D. A time-aware self-attention based neural network model for sequential recommendation. *Appl. Soft Comput.* **2023**, *133*, 109894. [CrossRef]
24. Zhang, X.; Lin, H.; Xu, B.; Li, C.; Lin, Y.; Liu, H.; Ma, F. Dynamic intent-aware iterative denoising network for session-based recommendation. *Inf. Process. Manag.* **2022**, *59*, 102936. [CrossRef]
25. Dang, Y.; Yang, E.; Guo, G.; Jiang, L.; Wang, X.; Xu, X.; Sun, Q.; Liu, H. Uniform sequence better: Time interval aware data augmentation for sequential recommendation. In Proceedings of the AAAI Conference on Artificial Intelligence, Washington, DC, USA, 7–14 February 2023; Volume 37, pp. 4225–4232.
26. Yue, Z.; Wang, Y.; He, Z.; Zeng, H.; McAuley, J.; Wang, D. Linear recurrent units for sequential recommendation. In Proceedings of the 17th ACM International Conference on Web Search and Data Mining, Mérida, Mexico, 4–8 March 2024; pp. 930–938.

27. Li, J.; Ren, P.; Chen, Z.; Ren, Z.; Lian, T.; Ma, J. Neural attentive session-based recommendation. In Proceedings of the 2017 ACM on Conference on Information and Knowledge Management, Singapore, 6–10 November 2017; pp. 1419–1428.
28. Jin, D.; Jin, Z.; Zhou, J.T.; Szolovits, P. Is bert really robust? a strong baseline for natural language attack on text classification and entailment. In Proceedings of the AAAI Conference on Artificial Intelligence, Singapore, 20–27 January 2020; Volume 34, pp. 8018–8025.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.